

EOG-Based Assistive Navigation System

Project Work Plan | Revised Scope | 2025

Confidential — Internal Use Only

Team: Jatinderpal Singh | Sambhav Manhas

Revision Notice: Claude API dependency removed. Section 4 (AI Decision Layer) now uses a **local rule-based decision engine** — no API key required. All other sections unchanged.

1. Project Overview

An EOG (Electrooculography) based assistive navigation system that allows users to control a computer entirely through eye signals — blinks and winks. The system replaces traditional motor imagery EEG with reliable, affordable eye-movement detection and integrates a **local rule-based decision engine** to intelligently interpret signals and execute OS-level navigation actions (**no external API required**).

This is a pivot from the original motor imagery BCI concept. The new system is fully achievable with existing hardware and produces a working, demonstrable product.

2. What Is New in This Version

2.1 Navigation via EOG signals only

The system no longer attempts motor imagery (EEG). All control is through eye-based EOG signals which are 10–50× stronger and far more reliable with consumer-grade hardware.

2.2 Local rule-based decision engine (replaces Claude API)

✓ Instead of calling the Claude API, a lightweight local Python function (`local_decision.py`) maps each detected signal directly to a navigation action. No internet connection, no API key, zero latency — the decision is instant and fully offline.

```
# local_decision.py
def decide_action(signal, highlighted_item):
    """signal: 'blink' | 'wink_left' | 'wink_right' | 'double_blink'"""
    if signal == 'blink':
        return f'open_{highlighted_item}'
    if signal == 'double_blink':
        return 'go_back'
    if signal == 'wink_right':
        return 'move_next'
    if signal == 'wink_left':
        return 'move_previous'
    return 'no_action'
```

2.3 Scanning interface for navigation

A custom scanning UI cycles through navigation options (folder, browser, camera, notepad, image viewer, etc.) automatically. The user blinks to select the highlighted item — no direction signal required.

2.4 Full OS navigation capability

The system can open folders, images, camera, browser, and other applications using Python subprocess calls — fulfilling the complete navigation requirement specified by the project supervisor.

3. Signal Mapping

Signal	EOG Detection	Navigation Action	Reliability
--------	---------------	-------------------	-------------

Single blink	Large spike on Fp1, both eyes	Select highlighted item	High
Double blink	Two spikes within 600 ms	Go back / cancel	High
Wink left	Asymmetric deflection, left side	Move highlight to previous	Medium
Wink right	Asymmetric deflection, right side	Move highlight to next	Medium

4. New Data Collection Plan

4.1 Electrode placement (new)

- IN+ on Fp1 (left forehead, above left eyebrow)
- IN– on left earlobe (referential setup, not bipolar)
- GND on right earlobe

This referential Fp1 setup captures clean unipolar EOG signals and clearly distinguishes left wink, right wink, and bilateral blink.

4.2 Data collection protocol

- Run test_blink_wink.py — subject 002 (new session, separate from old data)
- 75 trials total: 25 blinks, 25 wink-left, 25 wink-right
- Pygame window guides the user with eye icons on screen
- Rest 5 seconds between each block to reduce eye fatigue
- Collect minimum 2 sessions on different days for robustness

4.3 Data quality checks

- Inspect raw signal in real time — blink peaks should exceed 3× baseline amplitude
- Reject trials where electrode contact was poor (flat or noisy signal)
- Ensure DC offset is accounted for before peak detection threshold is set

4.4 Model retraining

- Train new eog_1ch_model.pkl using train_eog_1ch.py on fresh Fp1 data
- Target: ≥80% cross-validation accuracy before integration
- Test live with mouse_control_eog.py in simulate mode first, then real hardware

5. Phase-wise Work Plan

Phase	Task	Key Deliverable	Status
Phase 1	New electrode setup + data collection (75 trials total)	Fp1-labeled EOG trials saved as subject_002	Pending
Phase 2	Train new 1-channel EOG classifier	eog_1ch_model.pkl with ≥80% accuracy	Pending
Phase 3	Build scanning navigation UI	Python UI that cycles through 6 OS actions	In progress
Phase 4	Implement local decision engine (replaces Claude API)	local_decision.py — signal + item → action, fully offline	✓
Phase 5	Connect EOG model output to navigation pipeline	Full pipeline: blink → local_decision() → open menu	Pending
Phase 6	Demo preparation + fallback keyboard control	Recorded demo video + keyboard backup for reading	Pending

■ Phase 4 updated: Claude API replaced with local_decision.py — no API key needed.

6. Technology Stack

Layer	Component	Detail
Hardware	BioAmp EXG Pill + Arduino R4 Minima	Single pill, Fp1 referential, COM7, 500 Hz
Signal processing	Python + scipy + numpy	Bandpass filter, peak detection, threshold classifier
ML model	SVM (eog_1ch_model.pkl)	3-class: blink / wink_left / wink_right
Decision layer	Local rule-based engine (local_decision.py)	Signal + highlighted item → action. Offline. No API key required.
Navigation UI	Python (tkinter or pygame scanning UI)	Auto-cycling highlight, 6 OS actions
OS execution	Python subprocess	Opens folder, browser, camera, notepad, image viewer

7. Demo Script for Examination

The demo will show a complete end-to-end navigation session without any manual mouse or keyboard input (except for the keyboard fallback if hardware fails).

Demo sequence

- Start scanning UI — 6 options highlighted one by one automatically
- Wink right twice → highlight moves to Open Browser
- Blink once → browser opens (Chrome/Edge)
- Wink left → highlight moves back to Open Folder
- Blink once → File Explorer opens
- Double blink → returns to main menu
- Wink right → highlight moves to Open Camera
- Blink once → Windows Camera app launches

Fallback plan

- Keyboard keys (B, W, X, D) simulate blink/wink signals for demo if hardware glitches
- Pre-recorded demo video as backup if live hardware fails entirely

8. Key Risks and Mitigations

Risk	Mitigation	Fallback
Poor electrode contact during demo	Wet scalp, press firmly, use tape to hold electrodes	Keyboard simulation mode
Wink detection unreliable	Switch to auto-scan mode — only blink needed	Auto-scan removes need for wink signals
local_decision.py gives wrong action	Add unit tests for all 4 signal × 6 item combinations	Direct hardcoded subprocess call as last resort
Insufficient training data	Collect 2+ sessions, augment with noise injection	Use pre-existing eog_3class_model.pkl

This document is a living plan — update phase statuses as work progresses. All previous motor imagery data and EEG-based models are deprecated and not part of this revised scope. Claude API dependency fully removed in this revision.